



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

Time Allotted : 3 hrs

Full Marks : 70

Figures out of the right margin indicate full marks.

*Candidates are required to answer Group A and
any 5 (five) from Group B to E, taking at least one from each group.*

Candidates are required to give answer in their own words as far as practicable.

	Group – A (Multiple Choice Type Questions)	
1.	Choose the correct alternative for the following:	10 x 1=10
(i)	In a 2D array <code>int arr[100][50]</code> , the base address is 100. Then what will be the address of <code>arr[50][30]</code> ? (a) 5162 (b) 5160 (c) 5100 (d) None of these	
(ii)	What is the output of the given function if the variable <code>start</code> points to the first node of the following linked list? <code>0->1->2->3->4->5->6->7->8->9->10</code> <pre>void fun(struct node* start) { if(start != NULL) { if(start->next != NULL&& start->next->next != NULL) fun(start->next->next->next); printf("%d ", start->data); } }</pre> (a) 0 2 4 6 8 10 (b) 10 8 6 4 2 0 (c) 0 6 9 3 (d) 9 6 3 0	

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

(iii)	The postfix equivalent of $*+ab-cd$ is (a) $ab+cd-*$ (b) $ab+cd*-$ (c) $abcd+-*$ (d) $ab+-cd*$	
(iv)	If a connected undirected graph has 10 vertices, then its spanning tree (a tree that spans over all the vertices) has: (a) 9 vertices and 9 edges (b) 9 vertices and 10 edges (c) 10 vertices and 9 edges (d) 10 vertices and 10 edges	
(v)	Linked Lists are not suitable for implementing (a) Insertion Sort (b) Binary Search (c) Linear Search (d) Selection Sort	
(vi)	Secondary clustering is a result of (a) Linear Probing (b) Separate Chaining (c) Quadratic Probing (d) Double Hashing	
(vii)	In an AVL tree of height 4. Determine the minimum no. of nodes present in the tree? (a) 10 (b) 11 (c) 12 (d) 13	

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

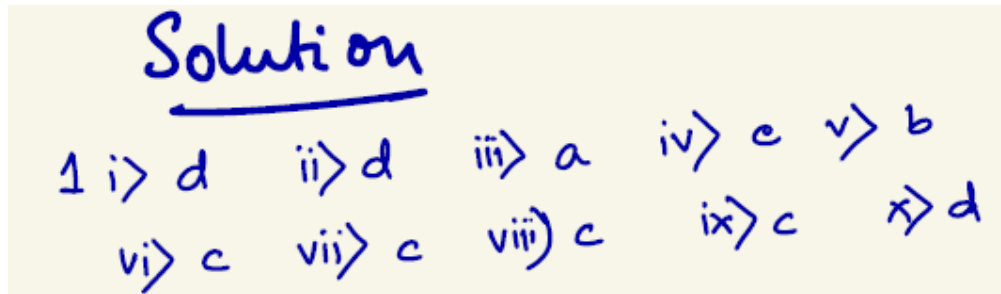
Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

(viii)	Recursion uses the ____ data structure as it follows ____ ordering. (a) Stack, FIFO (b) Queue, LIFO (c) Stack, LIFO (d) Queue, FIFO	
(ix)	The time complexity of heap sort in worst case is a) $O(\log n)$ b) $O(n)$ c) $O(n \log n)$ d) $O(n^2)$	
(x)	Which of the following statements is correct for a circular singly linked list with only a start pointer? (a) Both insertion and deletion at the front end take $O(1)$ time (b) Only insertion at the front end takes $O(1)$ time (c) Only deletion from the front end takes $O(1)$ time (d) No insertion or deletion operation at either end is possible in $O(1)$ time	



Group - B

2 (a) What is a sparse matrix?

[(CO1) (Remember/LOCQ)]

Given the following 4×5 matrix:

0 0 6 0 0
0 0 0 0 0
4 0 2 0 0
0 3 0 7 0

Compute its triplet array equivalent.

[(CO1) (Compute/LOCQ)]

Please do not write questions below this line

Paper setter:

**HERITAGE INSTITUTE OF TECHNOLOGY**

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

Is it beneficial to store this matrix as a triplet array? Justify your answer.

[(CO1) (Contrast/IOCQ)]

- (b) Given a singly-linked list with an odd number of elements, develop a pseudo-code or C-code to split it into two nearly equal sub-lists — one for the front half, and the other for the back half. The extra element should go in the front list.

[(CO3) (Develop/HOCQ)]

- (c) If for an algorithm, $f(n) = 3n^2 + 10$, state the worst case asymptotic time complexity of the algorithm. Show that it follows from the definition of worst case asymptotic time complexity.

[(CO1) (Apply/IOCQ)]

(1+3+2) + (3.5) + (0.5 + 2) = 12

2a) Sparse matrix is a matrix with at least $2/3$ rd of its elements as 0.

Triplet

4	5	5
0	2	6
2	0	4
2	2	2
3	1	3
3	3	7

Size of triplet = $6 \times 3 = 18$

" " original matrix = $4 \times 5 = 20$

$18 < 20 \therefore$ Beneficial

b)

```

fast = slow = start;
while (fast && fast->next)
{
    fast = fast->next->next;
    slow = slow->next;
}
start2 = slow->next;

```

c) $f(n) = 3n^2 + 10$

considering $g(n) = n^2$

$$3n^2 + 10 \leq c \cdot n^2$$

if $c = 3$,

$$3n^2 + 10 \not\leq 3n^2$$

$\therefore c = 4$

$$3n^2 + 10 \leq 4n^2$$

$$\Rightarrow n^2 \geq 10$$

$$\Rightarrow n \geq \sqrt{10}$$

$$\Rightarrow n_0 = \sqrt{10}$$

..

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

3 (a) State if the following statement is correct. Justify your answer.

$$O(n^2) = 2n^2 + 100$$

[(CO1) (Apply/IOCQ)]

(b) Construct an iterative function in C named reverse() that accepts the start node of a doubly-linked list as argument and reverses the list. Your function must reverse the actual orientation of the nodes. It should not simply reverse the values stored in the list. Ideally, reverse() should only need to make one pass of the list.

[(CO3) (Construct/HOCQ)]

(c) Given a 2D matrix of dimension 100 x 200 and also given that the base address of the 2D array is 2020. Assume that the array index of the matrix starts from [10, 20]. Find the address of the element with index [50, 160] for **column major** ordering.

Ans. This question does not mention the size of the data item stored in each location, so please allow the students to choose any value as they think appropriate.

[(CO2) (Understand/IOCQ)]

(3) + (6) + (3) = 12

3a) $f(n) = 2n^2 + 100$
 Let $g(n) = n^2$
 $2n^2 + 100 \leq cn^2$ min val. of $c = 3$
 for eqn to hold.
 $\therefore 2n^2 + 100 \leq 3n^2$
 $\Rightarrow n^2 \geq 100 \Rightarrow n \geq 10 \therefore n_0 = 10$

c) $B = 2020$ $i = 50$ $j = 160$
 $M = 90$ $N = 180$ $L_c = 20$ $L_r = 10$

Col. major address = $B + w \times [(j - L_c) * M + (i - L_r)]$

$= 2020 + 2 \times [(140 \times 90) + 40]$

$= 2020 + 2 \times [12640] = 2020 + 25280$

$= 27300$

if $w = 4$
 addr = 52580

Accept both

b) `temp = NULL;`
`curr = head;`
`while (curr != NULL)`
`{`
`temp = curr -> prev;`
`curr -> prev = curr -> next;`
`curr -> next = temp;`
`curr = curr -> prev;`
`}`
 // check for empty list
`if (temp != NULL)`
`head = temp -> prev;`

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

Group - C

- 4 (a) Why is it beneficial to use a circular array to implement a queue? Explain with an example.
[(CO4) (Remember/LOCQ)]
- (b) How is a circular linked list with a head pointer different from a circular linked list with a tail pointer? Which of the two would be better suited for implementing a queue and why? [(CO5)(Analyze/IOCQ)]
- (c) Stepwise show how the following infix expression is converted to its corresponding postfix expression.
 $((5+7)-(4*3))+(6/3)/2-9$
Next, Use a suitable data structure to evaluate the generated postfix expression.
[(CO3) (Apply/IOCQ)]

$$(2.5+1) + (2 + (0.5 + 1.5)) + (2.5+2) = 12$$

4. a say we consider 2 arrays of size 3 1 normal and 1 circular and use them as queues. For the following operations, they will behave as :-

	normal array	circular array						
	<table border="1"><tr><td></td><td></td><td></td></tr></table>				<table border="1"><tr><td></td><td></td><td></td></tr></table>			
Insert 5:	<table border="1"><tr><td>5</td><td></td><td></td></tr></table>	5			<table border="1"><tr><td>5</td><td></td><td></td></tr></table>	5		
5								
5								
Insert 6:	<table border="1"><tr><td>5</td><td>6</td><td></td></tr></table>	5	6		<table border="1"><tr><td>5</td><td>6</td><td></td></tr></table>	5	6	
5	6							
5	6							
Insert 7:	<table border="1"><tr><td>5</td><td>6</td><td>7</td></tr></table>	5	6	7	<table border="1"><tr><td>5</td><td>6</td><td>7</td></tr></table>	5	6	7
5	6	7						
5	6	7						
Delete :	<table border="1"><tr><td></td><td>6</td><td>7</td></tr></table>		6	7	<table border="1"><tr><td></td><td>6</td><td>7</td></tr></table>		6	7
	6	7						
	6	7						
Delete :	<table border="1"><tr><td></td><td></td><td>7</td></tr></table>			7	<table border="1"><tr><td></td><td></td><td>7</td></tr></table>			7
		7						
		7						
Insert 8: queue overflow		<table border="1"><tr><td>8</td><td></td><td>7</td></tr></table> insertion is still possible in circular array.	8		7			
8		7						

circular array is more memory efficient compared to normal array

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

b) CLL with head pointer will take $O(n)$ time complexity to insert node at both beginning and end and also $O(n)$ time complexity to delete first and last node. On the other hand a CLL with tail pointer will take $O(1)$ time complexity to insert nodes at the end and beginning. Also, deleting the 1st node will take $O(1)$ time. Only deleting the last node will take $O(n)$.

To implement a queue we go for insertion at the end and deletion from beginning. Hence CLL with tail ptr will be more opt.

c) Postfix expr will be $57+43*-63/2/9$ and evaluated value will be -8. Usage of stack, input, output and rule has to be specified. to get full marks.

- 5 (a) Implement (either pseudo-code or c-code is fine) the functionality of a queue using 2 stacks. You will only get full-marks if your implementation is efficient.

[(CO3) (Apply/IOCQ)]

Please do not write questions below this line

Paper setter:

**HERITAGE INSTITUTE OF TECHNOLOGY**

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

5 a) Enqueue :-
push every input element to the i/p stack

Dequeue :-
if (o/p stack is empty)
pop every element from i/p stack
and push them to o/p stack
until i/p stack is empty
pop from o/p stack

(b) Why is a Tail recursive function more efficient compared to a normal recursive function?

Ans. It is because the spaces of the local variables defined in the first call can be reused in every call as the scope of those variables of the earlier call ends before the next call is made. This is so because the recursive call is the last call in the function and hence the variable instances of the earlier call cannot be used after the next call is made. This saves a significant amount of overhead that is otherwise required whenever a function call is made.

[(CO1) (Remember/LOCQ)]

(c) An experienced Professor asked his students in the class to write a tail recursive function in C for a very popular series named after a famous Italian mathematician and asked them to display the 100th number of the series using it. One student just wrote a four-line function and left in five minutes while others were writing many lines of code for half an hour.

The Professor first ignored it thinking it to be some junk produced by an over-smart HITan. But after going to his room he found that it displays the correct number when he called it from his main(). He is still trying to figure out what happened. The codes are given below with some vital parts of the code being replaced by '?x'. If you just do not want to be such an unsuccessful Professor, then figure out what might have been written. You may just re-write the following few lines of code, filling out the eight missing ?x parts. Also, just write in max two sentences, why it works.

The student wrote like this -

```
int fiboTailRec(int n, int a, int b)
{
```

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

```

    if(?1 < ?2)
        return ?3;
    else
        return fiboTailRec (?4, ?5, ?6);
}

```

The Professor wrote like this -
fiboTailRec(100, ?7, ?8)

ANS. - int fiboTailRec(int n, int a, int b)

```

{
    if(n < 1)
        return a;
    else
        return fiboTailRec (n - 1, b, a + b);
}

```

The Professor wrote like this -
fiboTailRec(100, 0, 1)

Reason: First of all it is a tail recursive function as there is only one recursive call and it is the last call in the function. Also, basically we want the following pair assignment in each iteration (a, b) <- (b, a + b) and note that the function exactly does this in a reverse way, so if $F(k) = a$ and $F(k+1) = b$, then when after increment $k \leftarrow k + 1$, $F(k) = b$ and $F(k + 1) = a + b$.

[(CO3) (Apply/IOCQ)]
(4) + (2) + (5+1) = 12

Group - D

6 (a) What are the most well-known two ways of representing a graph? State their mutual benefits and drawbacks. State the time-complexity of Breadth First Search and what data structure does it use?

[(CO2) (Remember/LOCQ)]

(b) Construct a Binary Search Tree, an AVL tree and a B-Tree ($m = 2$) using the following keys.

[(CO5) (Apply/IOCQ)]

Which of the aforementioned data structures do you think will be the most efficient and why?

24, 98, 53, 46, 87, 50, 95

[(CO5) (Evaluate/HOCQ)]

(1 + 2 + (0.5 + 0.5)) + (1+ 3 + 2 + 2) = 12

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

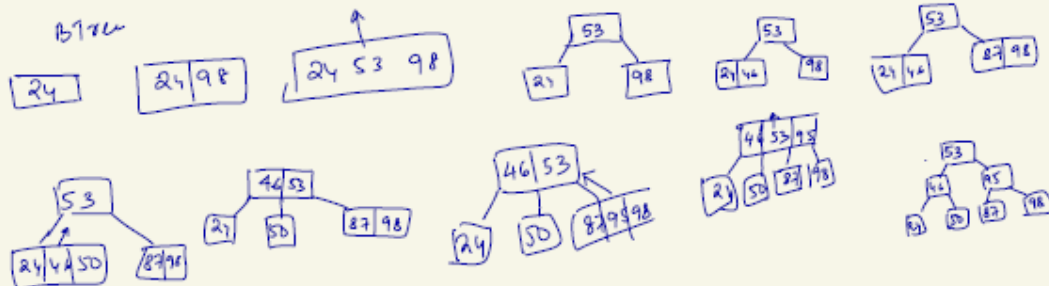
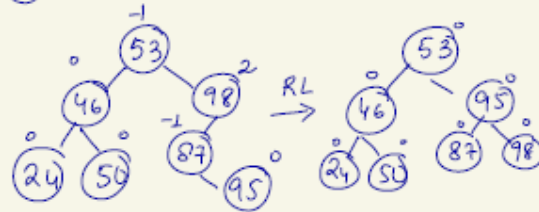
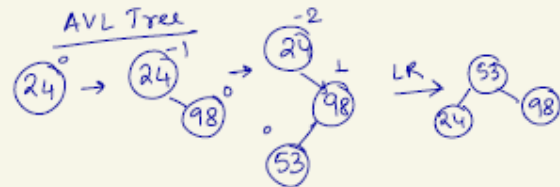
6a) Adj. List & Adj. Matrix

↓
applied when
higher no. of vertices
compared to edges.

↓
applied when higher
no. of edges compared
to higher no. of vertices.

BFS uses queue

↓
Complexity : $O(V+E)$ → Adj List
 $O(V^2)$ → Adj matrix



For RAM storage AVL tree will be
more efficient while for secondary storage
B+ Tree will reduce the read/write overhead.

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

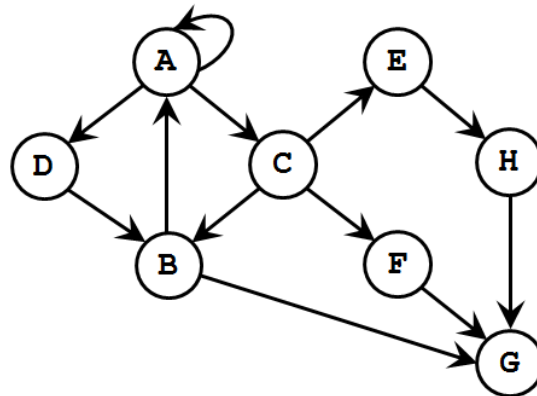
Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

7 (a) Perform depth-first traversal on the given graph, with A as the starting node. If more than one vertex can be visited at any given time, use alphabetical order to make the choice. Compute the discovery time and finishing time of each vertex and draw the depth-first tree. Identify and label all possible tree edges and non-tree edges in the original graph.



[(CO3) (Apply/IOCQ)]

(b) (i) Construct a Binary Search Tree using the following traversals.

Inorder: 7, 11, 20, 23, 30, 45, 56, 59, 60, 76, 92, 107, 112, 120

Preorder: 56, 23, 11, 7, 20, 30, 45, 92, 60, 59, 76, 112, 107, 120

[(CO5) (Construct/HOCQ)]

(ii) Now, consider deleting the following nodes in the given order: 23, 60, 56. Show how to do them by drawing the diagrams of the intermediate steps.

[(CO5) (Apply/IOCQ)]

(6) + (3+3) = 12

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

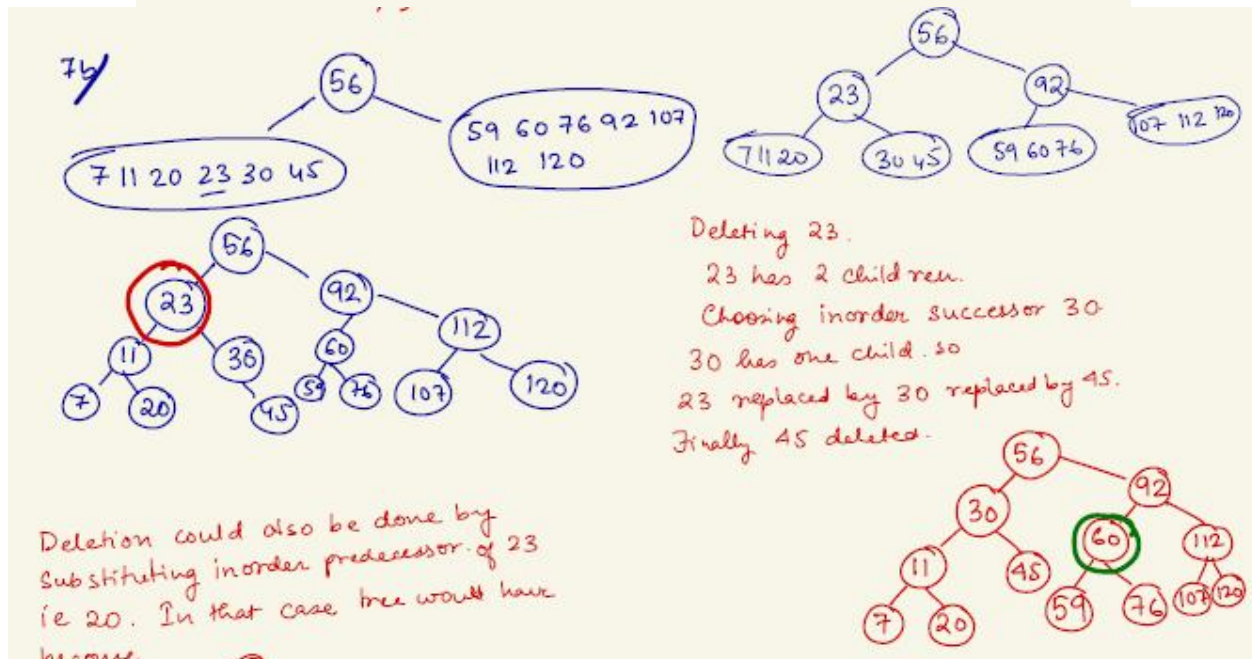
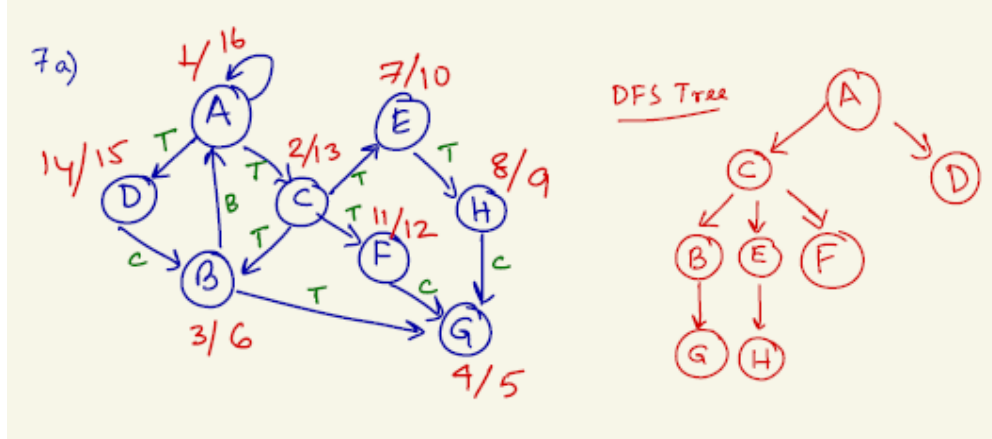
B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms



Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

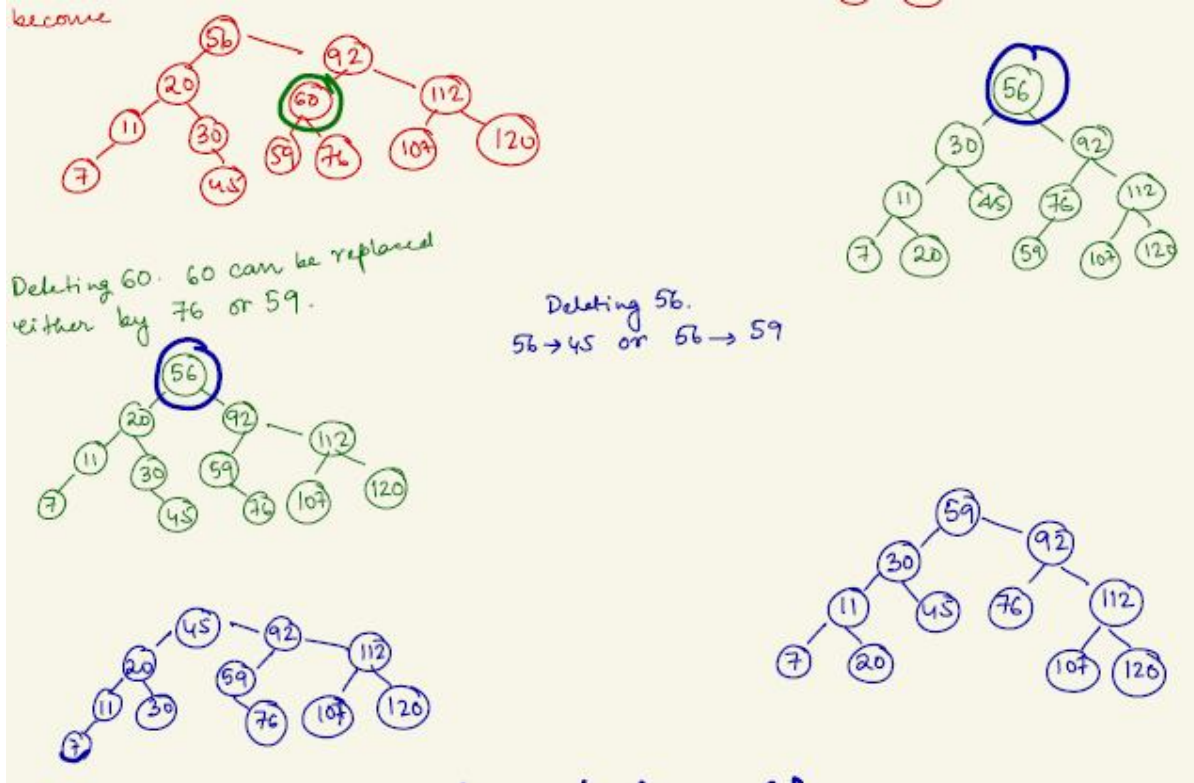
B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms



Group - E

8 (a) i) Write the pseudo-code for **Min-heapify**(A, i) routine, where A is an array containing integers and i is an index of the array. You **need not** implement Left(i) or Right(i).

[(CO2) (Understand/LOCQ)]

ANS. - Min_Heapify(A, i)

1. $l \leftarrow \text{LEFT}(i)$
2. $r \leftarrow \text{RIGHT}(i)$
3. if $(l < \text{heap_size}[A] \ \&\& \ A[l] < A[i])$ then
4. $\text{smallest} \leftarrow l$
5. else
6. $\text{smallest} \leftarrow i$
7. if $(r < \text{heap_size}(A) \ \&\& \ A[r] < A[\text{smallest}])$
8. then $\text{smallest} \leftarrow r$
9. if $(\text{smallest} \neq i)$
10. then $\text{exchange}(A[i] \leftrightarrow A[\text{smallest}])$

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

11. Min_Heapify(A, smallest)

ii) Suppose you are dealing with a heap and it is stored in an array A starting from the index 0 (like a C array). Write the C-codes for the functions Left(int i), Right(int i) and Parent(int i).

ANS. -

```
int Left(int i) {return 2*i + 1}
int Right(int i) {return 2*i + 2};
int Parent(int i) {return (i - 1)/2}    /* basically we need PARENT(j) =  $\lfloor (j-1)/2 \rfloor$  */
```

[(CO2) (Implement/LOCQ)]

iii) How will the codes change if the elements of the heap are stored compactly in the same array but starting from index 2, i.e., the root of the heap is stored in A[2]; A[0] and A[1] just contain some junk values?

```
int Left(int i) {return 2*(i-1) + 1} /* one may write 2*i -1 also */
int Right(int i) {return 2*(i-1) + 2}; /* one may write 2*i also */
int Parent(int i) {return (i + 1)/2} /* basically ceiling of i/2 */
```

[(CO6) (Analyze/HOCQ)]

(b) Fill up the table with asymptotic time complexities, some of them are shown for you –

	Best case	Average Case	Worst Case
Sequential Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(lg n)	O(lg n)
Interpolation Search	O(1)	O(lg lg n)	O(n)

[(CO4) (Summarising/LOCQ)]

(c) i) Suppose you are performing an insertion sort on the following elements stored in an array.
22, 29, 35, 42, 39, 25, 15, 32, 29

What will be the total number of comparisons required to sort them in ascending order? It is better to list the number of comparisons suffered by each element so that you may get some marks even if you do any mistake in evaluating the total count.

ANS. – 1 + 1 + 1 + 2 + 5 + 6 + 4 + 5 = 25 comparisons among array elements.

[(CO6) (Evaluate/IOCQ)]

Please do not write questions below this line

Paper setter:

**HERITAGE INSTITUTE OF TECHNOLOGY**

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

ii) Suppose in a certain type of search in an array of size n , after each comparison of the target with an array element, the search space becomes approximately cube root of n and after exactly k comparisons the search space reduces to 2 elements. What will be the asymptotic time complexity?

ANS.- We have $T(n) = 1 + T(n^{1/3})$, then $T(n^{1/3}) = 1 + T(n^{1/9})$ and so on.

Then $((n^{1/3})^{1/3})^{1/3} \dots k \text{ times} = 2 \Rightarrow n^{(1/3)(1/3)(1/3)\dots k \text{ times}} = 2$

Taking logarithm w.r.t. 2, we get $(1/3^k) \log_2 n = 1 \Rightarrow \log_2 n = 3^k \Rightarrow \log_3 (\log_2 n) = k$

Hence the time complexity is $O(\log \log n)$.

[(CO1) (Analyze/HOCQ)]

(3 + 1 + 2) + (2) + (2 + 2) = 12

9 (a) Do an average case analysis for finding the time complexity of Binary Search. Assume that the element you are trying to find is there in the array, i.e., the probability of not finding the element is zero.

[(CO6) (Analyze/IOCQ)]

ANS.

- Suppose when you first compare the key with the middle element, it matches, then the number of comparisons required to find the key is only 1. Number of positions for which only one comparison is required is therefore 1.
- Now if you do not find the key in the 1st comparison but you find it in the 2nd comparison, then number of such positions = 2 (middle elements of the half-arrays).
- The number of positions for which 3 comparisons are reqd. is 4.
- Hence average number of comparisons
- $\leq (1/n) [(1.1 + 2.2 + 4.3 + 8.4 + \dots + 2^{(\lg n - 1)} \lg n)] = S / n$ (say)

$$S = 1.1 + 2.2 + 4.3 + 8.4 + \dots + 2^{(\lg n - 1)} \lg n$$

$$2S = 2.1 + 4.2 + 8.3 + 16.4 + \dots + 2^{\lg n} \lg n$$

$$S - 2S = 1 + [2(2-1) + 4(3-2) + 8(4-3) + \dots] - 2^{\lg n} \lg n$$

$$-S = 1 + [2 + 4 + 8 + 16 + \dots + 2^{(\lg n - 1)}] - 2^{\lg n} \lg n$$

$$-S = 1 + [2(2^{(\lg n - 1)} - 1) / (2 - 1)] - n \lg n = 1 + 2(n/2 - 1) - n \lg n = 1 + n - 2 - n \lg n$$

$$\text{So, } S = n \lg n - n + 1.$$

$$\text{So, } T_{\text{avg}}(n) = (n \lg n - n + 1) / n = \lg n - 1 + 1/n = O(\lg n)$$

Please do not write questions below this line

Paper setter:

**HERITAGE INSTITUTE OF TECHNOLOGY**

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

(b) i) Suppose we are doing Radix Sort of 128 integers and the integers have hexadecimal digits, how would we lose if do not use Counting Sort as a sub procedure but use Heap Sort instead.

[(CO6) (Analyze/HOCQ)]

If we use counting sort, its complexity will be $O(n + k)$. Here $n = 128$, $k = 16$ (< 128) and so

$T(n) = O(n)$, whereas, for Heap Sort $T(n) = n \lg n$. Time complexity of radix sort is

$T(n) = (\text{no. of digits}) * (\text{time complexity of the sorting algorithm used as a sub-procedure})$

Hence, a $\lg n$ factor decrease in speed will be observed.

ii) Write the recurrence relation for the best case time complexity of Quick Sort and solve it by the iterative method.

[(CO4)(Analyze/IOCQ)]

$T(n) = 2T(n/2) + cn$ where c is a +ve constant.

$2T(n/2) = 2(2T(n/4) + c(n/2)) \Rightarrow 2T(n/2) = 2^2T(n/2^2) + cn$

.....

.....

$2^{k-1}T(n/2^{k-1}) = 2^kT(n/2^k) + cn = 2^kT(1) + cn = cn$, considering $2^{k-1} < n \leq 2^k$.

Adding all, we get $T(n) = cnk \leq c n \lceil \lg n \rceil = O(n \lg n)$

(c) i) Use 'linear probing with skipping' method to insert the following numbers in a table of size 6 in the order given: 76, 93, 40, 63, 28. Draw the final configuration of the Hash Table. Show the intermediate steps by briefly writing the calculations that happen at every stage.

Use the hash function $\text{floor}(h(k) + ic) \bmod 6$, where $h(k) = k \bmod 6$, $c = 1.4$ and $i = 0, 1, 2, 3, \dots$

[(CO6) (Apply/IOCQ)]

Ans.

ii) A compiler always needs a hash-table to do its operations. Name a job that a compiler performs where using a hash-table is natural and essential.

Ans. The names of the identifiers defined by the users is kept in a Hash table (commonly called Symbol table) as they need to be accessed in constant time, otherwise the compiler will be very slow.

[(CO5) (Remember/LOCQ)]

(4) + (2 + 2) + (3 + 1) = 12

Please do not write questions below this line

Paper setter:



CONFIDENTIAL

HERITAGE INSTITUTE OF TECHNOLOGY

B. Tech Semester Examination: 2021

Session : 2021-2022

Discipline : Computer Science and Engineering

Paper Code : CSEN2101

Paper Name : Data Structure & Algorithms

Cognition Level	LOCQ	IOCQ	HOCQ
Percentage distribution	21.35%	57.3%	21.35%

Course Outcome:

After the completion of this course students will be able to

CO 1: Understand and remember the basics of data structures and how time complexity analysis is applicable to different types of algorithms.

CO 2: Understand the significance and utility of different data structures and the context of their application. (For example, the queue in front of ticket counters uses first-in-first-out paradigm in a linear data structure)

CO 3: Apply different types of data structures in algorithms and understand how the data structures can be useful in those algorithms.

CO 4: Analyze the behaviour of different data structures in algorithms. (For example, given an algorithm that uses a particular data structure, how to calculate its space and time complexity.)

CO 5: Evaluate solutions of a problem with different data structures and thereby understand how to select suitable data structures for a solution. (For example, what are the different ways to find the second largest number from a list of integers and which solution is the best.)

CO 6: Evaluate different types of solutions (e.g. sorting) to the same problem.

*LOCQ: Lower Order Cognitive Question; IOCQ: Intermediate Order Cognitive Question; HOCQ: Higher Order Cognitive Question

Please do not write questions below this line

Paper setter: